Assignment 1

Calum Murphy

2025-02-17

Install necessary libraries

Part 1

Building the portfolio

For the portfolio I decided to select 10 different stocks from various different sectors.

```
stock_symbols <- c("AAPL", "MSFT", "JPM", "V", "TSLA", "WMT", "PFE", "NKE", "XOM", "NFLX")
```

Generating return data

```
#head(return_data)
stocks = lapply(stock_symbols, function(sym) {
  dailyReturn(na.omit(getSymbols(sym, src="yahoo", from="2018-02-18", to="2025-02-17", auto.assign=FALSE
})


return_data = do.call(merge, stocks)

colnames(return_data) <- stock_symbols
```

Split return data into a training set and test set

We will split the data into a training set which we will train the genetic algorithm on, and a test set which we will use to evaluate the genetic algorithm's performance on unseen ('future') data.

```
# 75% training set and 25% testing set
cutoff <- round(0.75*nrow(return_data)) #Row where we will cut off training set

training_return_data <- return_data[1:(cutoff-1),]
testing_return_data <- return_data[cutoff:nrow(return_data),]

testing_return_data[1,]
```

```
##                    AAPL        MSFT         JPM           V         TSLA
## 2023-05-15 -0.002897375 0.001585883 0.00842647 0.006180278 -0.009703475
##                     WMT          PFE          NKE          XOM        NFLX
## 2023-05-15 -0.007774237 -0.005086978 -0.003161112 -0.006712035 -0.01176851
```

We can see that the cutoff date is the 15th of May 2023. Everything before this is the training set and everything after, and including, this date is the testing set.

Calculate Mean daily returns of training set

```
mean_daily_returns_train <- apply(training_return_data, 2, mean)
mean_daily_returns_train
```

```
##         AAPL         MSFT          JPM            V         TSLA          WMT
## 0.0012717627 0.0011152867 0.0003210108 0.0006540725 0.0023935728 0.0004481401
##          PFE          NKE          XOM         NFLX
## 0.0001998132 0.0006497447 0.0004733254 0.0006070035
```

Covariance Matrix

```
train_cov_matrix <- cov(training_return_data) * 252
print(round(train_cov_matrix,6))
```

```
##           AAPL     MSFT      JPM        V     TSLA      WMT      PFE      NKE
## AAPL 0.109027 0.077914 0.048972 0.060118 0.102406 0.028557 0.029290 0.058429
## MSFT 0.077914 0.096173 0.046935 0.060711 0.091966 0.029155 0.030104 0.056892
## JPM  0.048972 0.046935 0.102192 0.056962 0.058123 0.020271 0.031394 0.053311
## V    0.060118 0.060711 0.056962 0.082842 0.070571 0.020539 0.030116 0.054896
## TSLA 0.102406 0.091966 0.058123 0.070571 0.434410 0.023227 0.016640 0.073463
## WMT  0.028557 0.029155 0.020271 0.020539 0.023227 0.051360 0.017936 0.022787
## PFE  0.029290 0.030104 0.031394 0.030116 0.016640 0.017936 0.066906 0.025032
## NKE  0.058429 0.056892 0.053311 0.054896 0.073463 0.022787 0.025032 0.107942
## XOM  0.038143 0.033970 0.064226 0.046882 0.043157 0.016934 0.025431 0.043063
## NFLX 0.075504 0.076399 0.035589 0.052417 0.117992 0.025951 0.022158 0.057663
##            XOM     NFLX
## AAPL 0.038143 0.075504
## MSFT 0.033970 0.076399
## JPM  0.064226 0.035589
## V    0.046882 0.052417
## TSLA 0.043157 0.117992
## WMT  0.016934 0.025951
## PFE  0.025431 0.022158
## NKE  0.043063 0.057663
## XOM  0.114444 0.023426
## NFLX 0.023426 0.220843
```

Examining the table we can see that covariance is generally in the porfolio which makes it suitable to train a genetic algorithm on.

Fitness Function

A function that will maximise the sharpe ratio, a measure that calculates the proportion of a return based on the risk taken.

```
fitness_function <- function(weights){
  weights <- weights / sum(weights)
  portfolio_returns <- (sum(mean_daily_returns_train*weights) + 1)^252 - 1 #Annualised returns
  portfolio_risk <- sqrt(t(weights) %*% (train_cov_matrix) %*% weights)
  sharpe_ratio <- portfolio_returns/portfolio_risk
  return(sharpe_ratio)
}
```

Genetic Algorithm

Since we are dealing with floating point data, the type of genetic algorithm we will use is "real-valued".

```
GA <- ga(
  type = "real-valued",
  fitness = fitness_function,
  lower=rep(0,10), # Minimum weights for portfolio
  upper=rep(1,10), # Maximum weights for portfolio
  popSize = 250,
  maxiter = 1000,
  pcrossover = 0.7,
  pmutation = 0.5
)

summary(GA)
```

```
## -- Genetic Algorithm -------------------
##
## GA settings:
## Type                  =  real-valued
## Population size       =  250
## Number of generations =  1000
## Elitism               =  12
## Crossover probability =  0.7
## Mutation probability  =  0.5
## Search domain =
##       x1 x2 x3 x4 x5 x6 x7 x8 x9 x10
## lower  0  0  0  0  0  0  0  0  0   0
## upper  1  1  1  1  1  1  1  1  1   1
##
## GA results:
## Iterations             = 1000
## Fitness function value = 1.363128
## Solution =
##             x1        x2          x3          x4        x5          x6
## [1,] 0.9643142 0.3488516 0.004166245 0.004176304 0.7678527 0.004420117
##               x7         x8          x9         x10
## [1,] 0.001221344 0.00465782 0.007548854 0.006263852
```

```
# Extract best weights
optimal_weights <- GA@solution / sum(GA@solution) # Makes sum of weights equal to 1
optimal_weights
```

```
##             x1        x2          x3          x4        x5        x6
## [1,] 0.4562699 0.1650608 0.001971279 0.001976038 0.3633132 0.0020914
##                x7         x8          x9         x10
## [1,] 0.0005778848 0.00220387 0.003571777 0.002963772
```

A fitness function value of >1 indicates that the portfolio is generating more return per risk taken - ideal!

Evaluating the optimal weights against random weights and equal weights

```
optimal_weights <- matrix(optimal_weights, ncol = 1) # Transforms solution into a vector of weights
glue('Optimal weights sharpe ratio: {round(fitness_function(optimal_weights),4)}')
```

```
## Optimal weights sharpe ratio: 1.3631
```

```
# Equal weights
equal_weights <- rep(1, 10)
glue('Equal rates sharpe ratio: {round(fitness_function(equal_weights),4)}')
```

```
## Equal rates sharpe ratio: 0.9628
```

```
# Random weights
random_sharpe_train <- 0
best_random_weights <- NULL


# Finds the best random weights on 1000 iterations

for (i in 1:1000) {

  random_weights_train <- runif(n = length(stocks))
  random_weights_train <- random_weights_train / sum(random_weights_train) #Ensure weights sum to 0

  sharpe_num = fitness_function(random_weights_train)

  if (sharpe_num > random_sharpe_train) {
    random_sharpe_train <- sharpe_num

    best_random_weights <- random_weights_train
  }
}

glue('Random weights sharpe ratio: {round(fitness_function(best_random_weights),4)}')
```

```
## Random weights sharpe ratio: 1.1884
```

Genetic algorithm sharpe ratio is higher, therefore it performs better against randomly generated weights and equal weights on the training set.

Evaluate GA performance on future data (i.e. the test set)

```
# We first have to calculate mean returns of the test set
mean_daily_returns_test <- apply(testing_return_data, 2, mean)

# Also calculate the covariance matrix
test_cov_matrix <- cov(testing_return_data) * 252

# Similar to fitness function, this function returns sharpe ratio based on test data
evaluate_weights <- function(weights) {
  weights <- weights / sum(weights)
  portfolio_returns_test <- (sum(mean_daily_returns_test * weights) + 1)^252 - 1 #Annualised returns of
  portfolio_risk_test <- sqrt(t(weights) %*% (test_cov_matrix) %*% weights)
  sharpe_ratio_test <- portfolio_returns_test/portfolio_risk_test
  return(sharpe_ratio_test)

}

glue('Optimal weights sharpe ratio on test data: {round(evaluate_weights(optimal_weights),4)}')
```

```
## Optimal weights sharpe ratio on test data: 1.5097
```

```
random_sharpe_test <- 0

best_weights_test <- NULL

# Finds the best random weights on 1000 iterations on the test set

for (i in 1:1000) {

  random_weights_test <- runif(n = length(stocks))
  random_weights_test <- random_weights_test / sum(random_weights_test)

  sharpe_num = evaluate_weights(random_weights_test)

  if (sharpe_num > random_sharpe_test) {
    random_sharpe_test <- sharpe_num
    best_weights_test <- random_weights_test
  }
}

glue('Equal weights sharpe ratio on test data: {round(evaluate_weights(equal_weights),4)}')
```

```
## Equal weights sharpe ratio on test data: 2.0712
```

```
glue('Random weights sharpe ratio on test data: {round(evaluate_weights(best_weights_test),4)}')
```

```
## Random weights sharpe ratio on test data: 3.794
```

The optimal weights from the genetic algorithm still performs well on the test data, however, both the randomly generated and the equal weighted portfolios perform better than the optimal weights. This suggests that the genetic algorithm is overfitting on the training set i.e. performing well on the training set but failing to generalise well on new, unseen data. I tried playing around with the parameters but to no real avail. Initially, the overfitting was a lot more extreme so I increased the dataset to include more years which did help the genetic algorithm to generalise better. Increasing the dataset further didn't seem to help. One possible way to mitigate this would be to train the genetic algorithm every 6 months so it can constantly adapt to market changes and generalise better.

Exploring different weightings of risk and return

For this section we will create two new fitness functions that will minimise risk and maximise returns and then train a genetic algorithm on these functions.

Maximising return

```
fitness_function_maximise_return <- function(weights){
  weights <- weights / sum(weights)
  portfolio_returns <- (sum(mean_daily_returns_train * weights) + 1)^252 - 1 #Annualised returns
  return(portfolio_returns) # Returns portfolio returns instead of sharpe ratio
  }

GA_maximise_return <- ga(
  type = "real-valued",
  fitness = fitness_function_maximise_return,
  lower=rep(0,10), # Minimum weights for portfolio
  upper=rep(1,10), # Maximum weights for portfolio
```

```
  popSize = 100,
  maxiter = 1000
)

summary(GA_maximise_return)
```

```
## -- Genetic Algorithm ------------------
##
## GA settings:
## Type                  =  real-valued
## Population size       =  100
## Number of generations =  1000
## Elitism               =  5
## Crossover probability =  0.8
## Mutation probability  =  0.1
## Search domain =
##       x1 x2 x3 x4 x5 x6 x7 x8 x9 x10
## lower  0  0  0  0  0  0  0  0  0   0
## upper  1  1  1  1  1  1  1  1  1   1
##
## GA results:
## Iterations             = 1000
## Fitness function value = 0.7670677
## Solution =
##               x1          x2         x3         x4        x5         x6
## [1,] 0.008291196 0.001585116 0.01040301 0.01429592 0.9863742 0.00707819
##              x7          x8         x9         x10
## [1,] 0.00404632 0.007602522 0.01551546 0.009443871
```

```
maximise_return_weights <- GA_maximise_return@solution / sum(GA_maximise_return@solution) # Makes sum o
maximise_return_weights <- matrix(maximise_return_weights, ncol = 1)

glue('Sharpe ratio of maximised return portfolio: {round(fitness_function(maximise_return_weights),4)}')
```

```
## Sharpe ratio of maximised return portfolio: 1.2408
```

A fitness function value of >0.7 means that the expected returns of the portfolio is more than 70% a year which is very high. With a return rate this high, we expect high volatilty. The calculated sharpe ratio (using the fitness function defined above) is above 1. This suggests that the possible return may be worth the associated risk with this portfolio.

Minimising risk

```
fitness_function_minimise_risk <- function(weights){
  weights <- weights / sum(weights)
  portfolio_risk <- sqrt(t(weights) %*% (train_cov_matrix) %*% weights)
  return(-portfolio_risk) # Returning negative value will ensure GA finds the smallest risk
}

GA_minimise_risk <- ga(
  type = "real-valued",
  fitness = fitness_function_minimise_risk,
  lower=rep(0,10), # Minimum weights for portfolio
  upper=rep(1,10), # Maximum weights for portfolio
  popSize = 100,
```

```
  maxiter = 1000
)

summary(GA_minimise_risk)
```

```
## -- Genetic Algorithm -------------------
##
## GA settings:
## Type                  =  real-valued
## Population size       =  100
## Number of generations =  1000
## Elitism               =  5
## Crossover probability =  0.8
## Mutation probability  =  0.1
## Search domain =
##       x1 x2 x3 x4 x5 x6 x7 x8 x9 x10
## lower  0  0  0  0  0  0  0  0  0   0
## upper  1  1  1  1  1  1  1  1  1   1
##
## GA results:
## Iterations             = 1000
## Fitness function value = -0.1837867
## Solution =
##              x1         x2         x3        x4         x5        x6        x7
## [1,] 0.01286344 0.01201873 0.03138671 0.1351145 0.01030816 0.9975375 0.6187795
##              x8        x9        x10
## [1,] 0.1213816 0.2244573 0.06661793
```

```
minimise_risk_weights = GA_minimise_risk@solution / sum(GA_minimise_risk@solution)
minimise_risk_weights <- matrix(minimise_risk_weights, ncol = 1)

glue('Sharpe ratio of minimising risk portfolio: {round(fitness_function(minimise_risk_weights),4)}')
```

```
## Sharpe ratio of minimising risk portfolio: 0.6157
```

The genetic algorithm looks to maximise the fitness function, so setting the fitness funcion to return a negative value ensured we were able to find a portfolio with very low risk. The risk is fairly low, 15-20%, but with risk this low we can't expect the returns to be that great. Calculating the sharpe ratio we can see that it is below 1 which tells us that this selection of weights may not be worth it as the returns aren't great enough to justify such minimal risk.

We can also see how the different weights perform on the test data

```
glue('Sharpe ratio of maximised return portfolio on test data: {round(evaluate_weights(maximise_return_
```

```
## Sharpe ratio of maximised return portfolio on test data: 1.4141
```

```
glue('Sharpe ratio of minimising risk portfolio on test data: {round(evaluate_weights(minimise_risk_wei
```

```
## Sharpe ratio of minimising risk portfolio on test data: 1.5875
```

On future data both portfolios perform well.

#Part 2

For this part we will select a portfolio from a pool of 50 stocks. The selection criteria I have decided to use is biggest mean daily return.

```r
#Loads stok data using tidyquant
pool_stocks <- tq_index("SP500")$symbol
```

```
## Getting holdings for SP500
```

```r
new_stocks_symbols <- sample(pool_stocks, 50)

getSymbols(new_stocks_symbols, src="yahoo", from="2017-02-18", to="2023-05-14") #Cutoff date of trainin
```

```
## Warning: Failed to open
## 'https://query2.finance.yahoo.com/v8/finance/chart/GEV?period1=1487376000&period2=1684022400&interval
## The requested URL returned error: 400
```

```
## Warning: Unable to import "GEV".
## cannot open the connection
```

```
##  [1] "VICI" "URI"  "BEN"  "ULTA" "AMAT" "AMP"  "TDG"  "CPAY" "FAST" "CAG"
## [11] "BMY"  "TTWO" "WAT"  "PSA"  "MSI"  "TRGP" "GE"   "EVRG" "ADSK" "PFG"
## [21] "ADP"  "VMC"  "MKC"  "MTCH" "IVZ"  "CRWD" "RVTY" "NCLH" "PHM"  "OMC"
## [31] "HIG"  "JCI"  "MLM"  "NDSN" "INTC" "WMT"  "PARA" "GWW"  "GOOG" "STLD"
## [41] "ORCL" "BIIB" "SYF"  "ACN"  "META" "BKNG" "FANG" "BR"   "WMB"
```

```r
new_stocks = lapply(new_stocks_symbols, function(sym) {
  dailyReturn(na.omit(getSymbols(sym, from="2017-02-18", to="2025-02-17", auto.assign=FALSE)))
})
new_stock_data = do.call(merge, new_stocks)

colnames(new_stock_data) <- new_stocks_symbols


#Gets rid of NaN values
new_stock_data <- new_stock_data[ , colSums(is.na(new_stock_data))==0]
```

Fitness function

This fitness function will maximise the mean of porfolio returns. (For this part I really struggled to keep the selection at 10 stocks. I tried to penalise portfolios that exceeded ten but I ended up just getting a value of 0 generated by the GA.)

```r
evaluate_stocks <- function(selection) {

  selected_stocks <- new_stock_data[, selection == 1]
  portfolio_return <- apply(selected_stocks, 2, mean)
  #Returns portfolio with best average return

  return(mean(portfolio_return))
}


ga_stocks <- ga(
  type = "binary",
  fitness = evaluate_stocks,
  nBits = ncol(new_stock_data),
  popSize = 50,
  maxiter = 200
)
```

```r
summary(ga_stocks)
```

```
## -- Genetic Algorithm ------------------
##
## GA settings:
## Type                  =  binary
## Population size       =  50
## Number of generations =  200
## Elitism               =  2
## Crossover probability =  0.8
## Mutation probability  =  0.1
##
## GA results:
## Iterations             = 200
## Fitness function value = 0.001132219
## Solution =
##      x1 x2 x3 x4 x5 x6 x7 x8 x9 x10  ...  x46 x47
## [1,]  1  0  0  1  0  0  0  0  0   0         0   0
```

```r
stocks_to_select = ga_stocks@solution
```

Creating stock data on assets identified by genetic algorithm

```r
stocks_to_select <- matrix(stocks_to_select, ncol = 1)
new_stock_data <- new_stock_data[,stocks_to_select==1]
```

#Calculating new mean returns

```r
new_mean_returns <- apply(new_stock_data, 2, mean)
```

Calculating new covariance matrix

```r
new_cov_matrix <- cov(new_stock_data) * 252
```

This fitness function is almost identical to the one defined above but instead takes new_mean_returns to calculate the portfolio returns.

```r
new_fitness_function <- function(weights){
  weights <- weights / sum(weights)
  portfolio_returns <- (sum(new_mean_returns*weights) + 1)^252 - 1 #Annualised returns
  portfolio_risk <- sqrt(t(weights) %*% (new_cov_matrix) %*% weights)
  sharpe_ratio <- portfolio_returns/portfolio_risk
  return(sharpe_ratio)
}
```

Repeat genetic algorithm for new portfolio

```r
new_GA <- ga(
  type = "real-valued",
  fitness = new_fitness_function,
  lower=rep(0, ncol(new_stock_data)), # Minimum weights for portfolio
  upper=rep(1,ncol(new_stock_data)), # Maximum weights for portfolio
  popSize = 10,
  maxiter = 1000,
```

```
  pcrossover = 0.7,
  pmutation = 0.5
)

summary(new_GA)
```

```
## -- Genetic Algorithm -------------------
##
## GA settings:
## Type                  =  real-valued
## Population size       =  10
## Number of generations =  1000
## Elitism               =  1
## Crossover probability =  0.7
## Mutation probability  =  0.5
## Search domain =
##       x1 x2 x3 x4
## lower  0  0  0  0
## upper  1  1  1  1
##
## GA results:
## Iterations             = 1000
## Fitness function value = 1.259131
## Solution =
##             x1         x2        x3        x4
## [1,] 0.2742497 0.02875995 0.9342813 0.3682161
```

```
new_optimal_weights = new_GA@solution
new_optimal_weights = new_optimal_weights/sum(new_optimal_weights)


new_optimal_weights = matrix(new_optimal_weights, ncol=1)
```

Evaluating new portfolio against old one

```
new_optimal_weights <- matrix(new_optimal_weights, ncol=1)

glue('Old portfolio sharpe ratio: {round(fitness_function(optimal_weights),4)}')
```

```
## Old portfolio sharpe ratio: 1.3631
```

```
glue('New portfolio sharpe ratio: {round(new_fitness_function(new_optimal_weights),4)}')
```

```
## New portfolio sharpe ratio: 1.2591
```

~~The new portfolio is performing better thanks to the genetic algorithm~~.